

- Content management systems
- Blogging platforms
- Systems that maintain counters
- Services that have expiring usage
- Systems that require heavy write requests (like log aggregators)

Column store databases should be avoided if you have to use complex querying or if your querying patterns frequently change. Also avoid them if you don't have an established database requirement, a trend which we are beginning to see in new systems.

Examples of column store NoSQL databases are Cassandra and Apache Hadoop HBase.

Graph base NoSQL database

Graph databases are basically built upon the Entity - Attribute - Value model. Entities are also known as nodes, which have properties. It is a very flexible way to describe how data relates to other data. Nodes store data about each entity in the database, relationships describe a relationship between nodes, and a property is simply the node on the opposite end of the relationship. Whereas a traditional database stores a description of each possible relationship in foreign key fields

or junction tables, graph databases allow for virtually any relationship to be defined on-the-fly.

Use cases: Graph base NoSQL databases are usually used in:

- Fraud detection
- Graph based search
- Network and IT operations
- Social networks, etc

Examples of graph base NoSQL databases are Neo4j, ArangoDB and OrientDB. 

References

- [1] <http://basho.com/resources/nosql-databases/>
- [2] <http://www.algoworks.com/blog/nosql-database-types/>
- [3] <https://www.thoughtworks.com/insights/blog/nosql-databases-overview>

By: Roopendra Vishwakarma

The author is passionate about researching on new technologies in DevOps and Web development. He has written many articles around various technologies, open source software, Web development and DevOps tools. He can be reached at roopendra@techieroop.com.

Continued from Page 71...

- Books authored can grow out of bounds.
- Avoiding repetition of data.

Many-to-many with Embedded relationship: It is not advisable to implement the many-to-many strategy with Embedded relationships, as it leads to unnecessary repetition of data and applications have to write complex logic to perform updates and retrievals.

Many-to-many with References relationship: In this strategy, collections are Referenced where they have a many-to-many relationship with each other.

An example of a many-to-many Referenced relationship between a bank account and account holders is illustrated here. The bank account document uses arrays to reference the account holders, and the account holders' document refers to the bank account using arrays.

```
{
  "_id": "123456789",
  "accountNumber": "123456789",
  "accountName": "ManytoManyAccount",
  "accountHolders": [ 111111111, 222222222 ]
}
```

```
{
  "_id": "111111111",
  "name": "Ritesh",
  "age": 25,
```

```
"phone": "9999999999",
"email": "abc@abc.com",
"account_id": [ 123456789 ]
}

{
  "_id": 222222222,
  "name": "Sohan",
  "age": 30,
  "phone": "8888888888",
  "email": "xyz@xyz.com",
  "account_id": [ 123456789 ]
}
```

Many-to-many Referenced relationships should be used when:

- Many-to-many relationships exist between entities.
- Both collections can grow out of bounds.
- Avoiding repetition of data. 

By: Ritesh Modi

The author is a technology architect and senior technology evangelist with Microsoft, with 14+ years of experience in the industry. He blogs at <http://automationnext.wordpress.com> and can be followed on Twitter @automationnext.